

4 Efficient Sorting

Efficient Algorithms · Wintersemester 2026–27

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
version 2026-11-03 14:09

Learning Outcomes

Unit 4: ***Efficient Sorting***

1. Know principles and implementation of *mergesort* and *quicksort*.
2. Know properties and *performance characteristics* of mergesort and quicksort.
3. Know the comparison model and understand the corresponding *lower bound*.
4. Understand *counting sort* and how it circumvents the comparison lower bound.
5. Know ways how to exploit *presorted* inputs.

Outline

4 Efficient Sorting

4.1 Mergesort
4.2 Quicksort
4.3 Comparison-Based Lower Bound
4.4 Integer Sorting
4.5 Adaptive Sorting
4.6 Python's list sort

Why study sorting?

- fundamental problem of computer science that is still not solved
- building brick of many more advanced algorithms
  - for preprocessing
  - as subroutine
- playground of manageable complexity
  to practice algorithmic techniques

Algorithm with optimal #comparisons in worst case?

Here:

- "classic" fast sorting method
- exploit **partially sorted** inputs

Part I

The Basics

Rules of the game

- **Given:**
  - array $A[0..n) = A[0..n-1]$ of n objects
  - a total order relation $\leq$ among $A[0], \ldots, A[n-1]$
    (a comparison function)
    Python: elements support <= operator (`__le__()`)
    Java: `Comparable` class (`x.compareTo(y) <= 0`)
- **Goal:** rearrange (i. e., permute) elements within A,
  so that A is *sorted*, i. e., $A[0] \leq A[1] \leq \cdots \leq A[n-1]$

- for now: A stored in main memory (*internal sorting*)
  single processor (*sequential sorting*)

Clicker Question



What is the complexity of sorting? Type you answer, e. g., as "`Theta(sqrt(n))`"

lower bound: cf. $\Omega(n)$ time for subarray sum
upper bound (algorithm's Θ-class of running time)

$\Theta(n \log n)$



→ *sli.do/effalg*

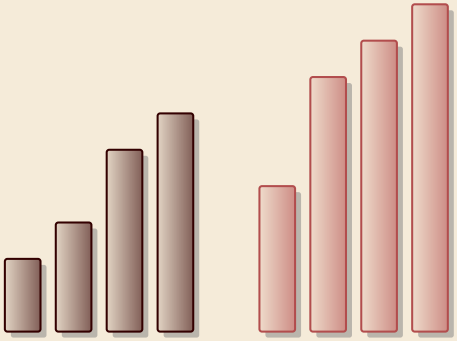
4.1 Mergesort

Merging sorted lists



Merging sorted lists

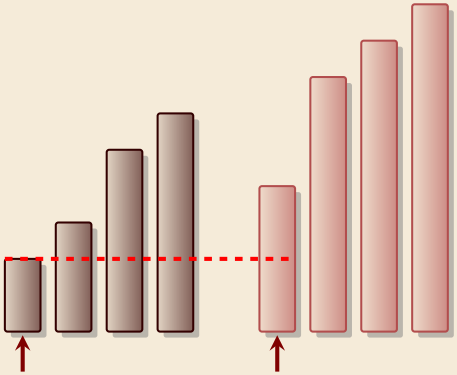




run1 run2 result

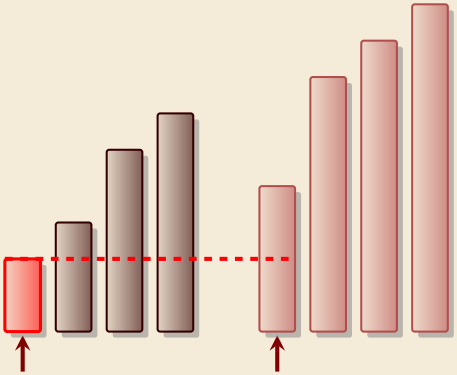
Merging sorted lists





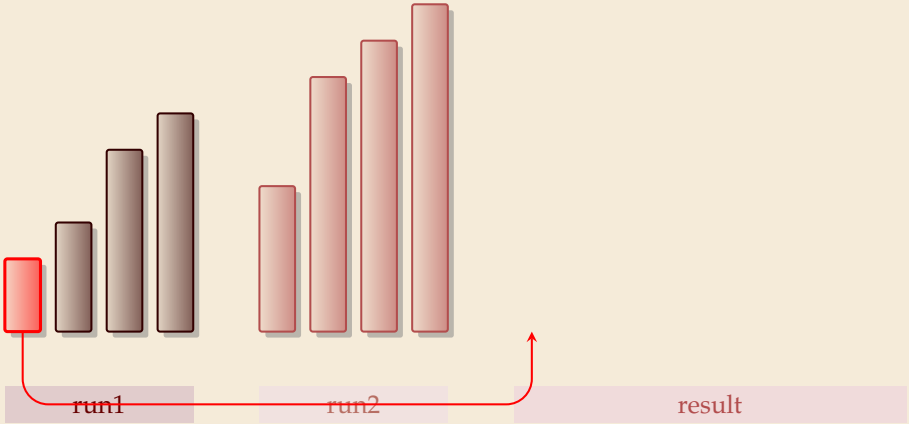
Merging sorted lists





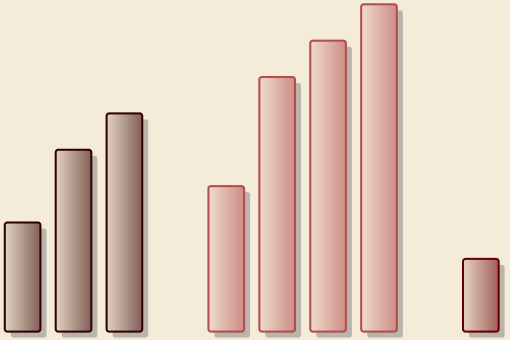
Merging sorted lists





Merging sorted lists





run1 run2 result

Merging sorted lists





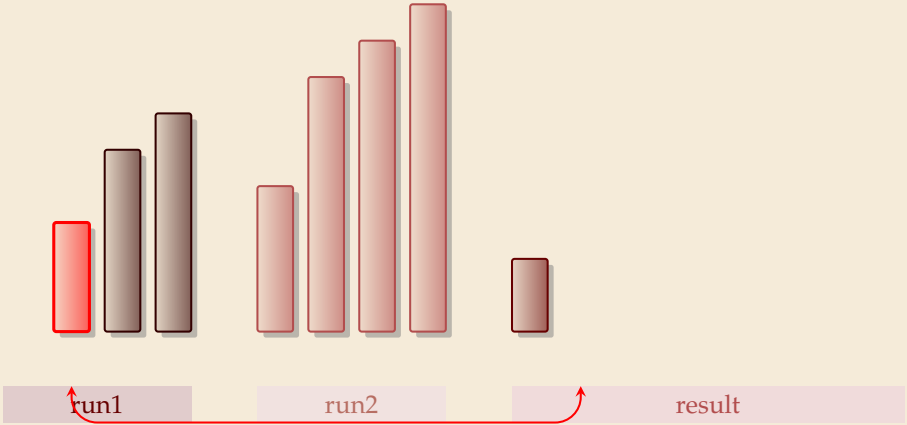
Merging sorted lists





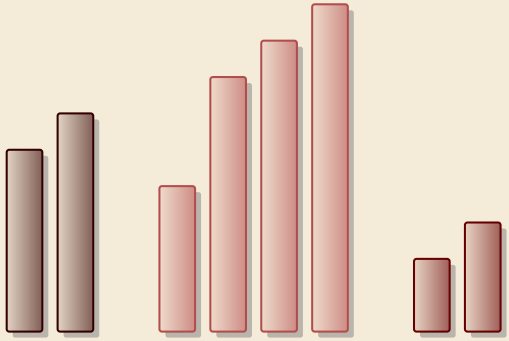
Merging sorted lists





Merging sorted lists





run1 run2 result

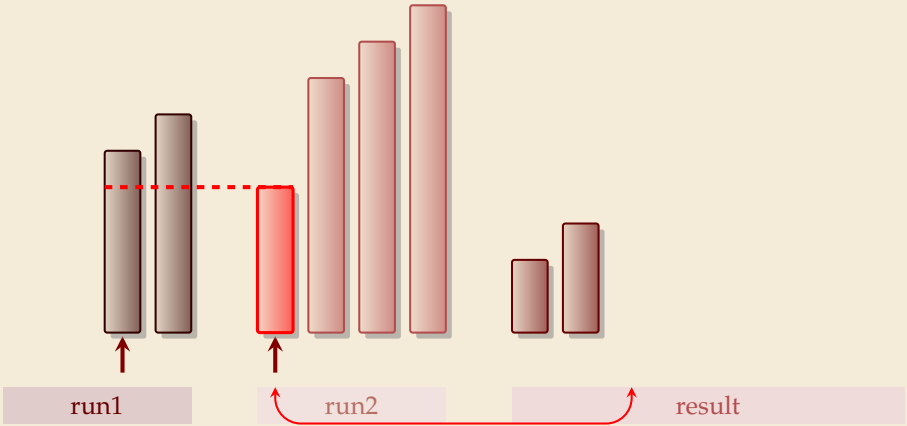
Merging sorted lists





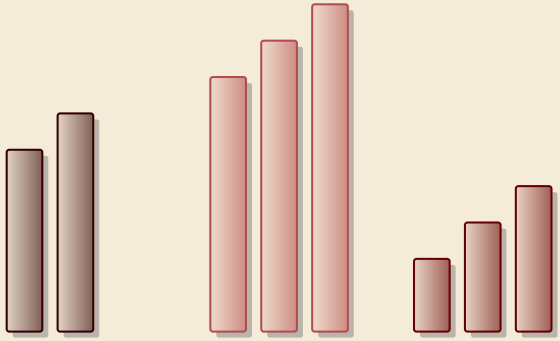
Merging sorted lists





Merging sorted lists

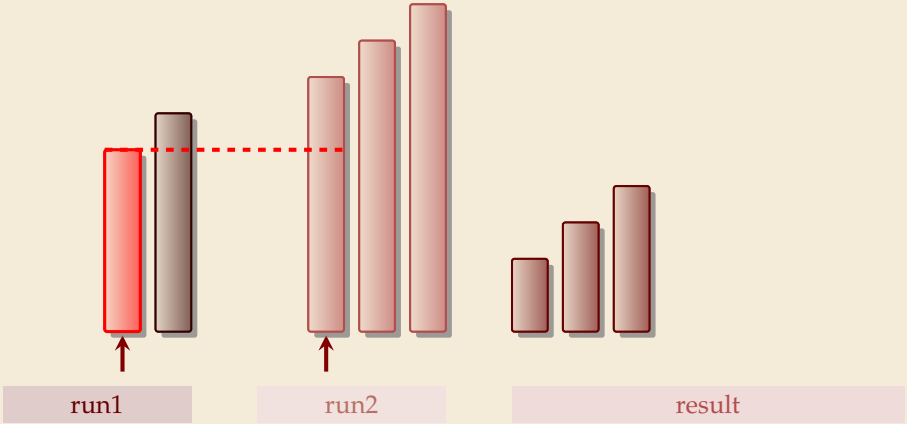




run1 run2 result

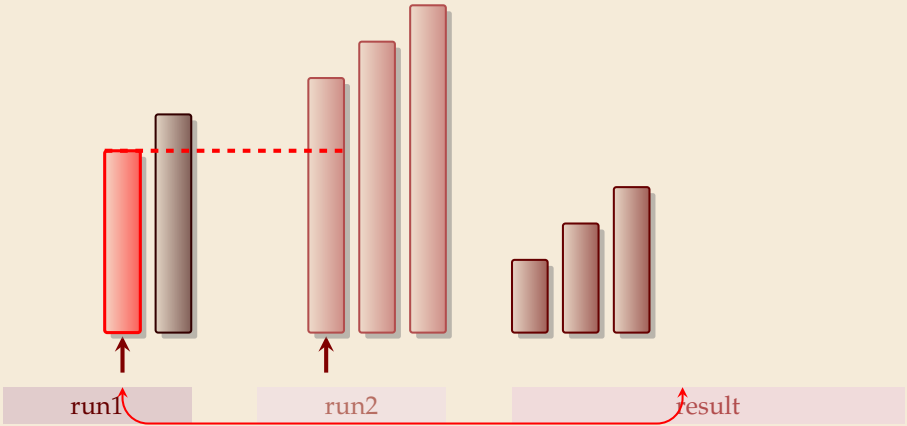
Merging sorted lists





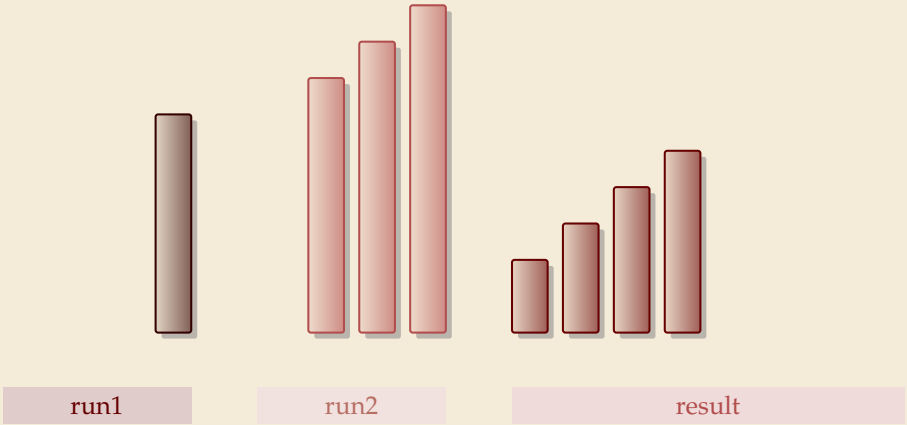
Merging sorted lists





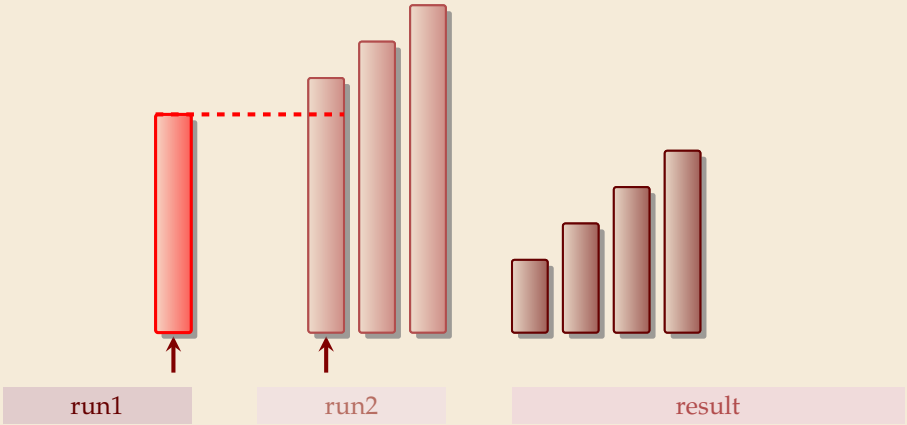
Merging sorted lists





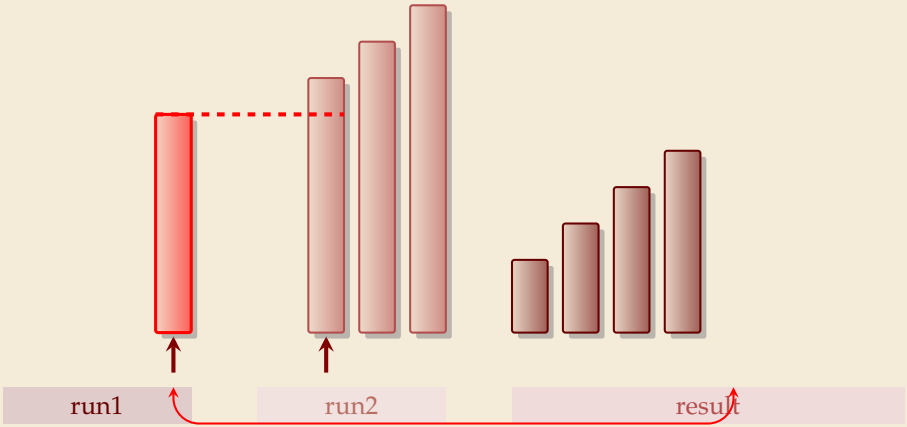
Merging sorted lists





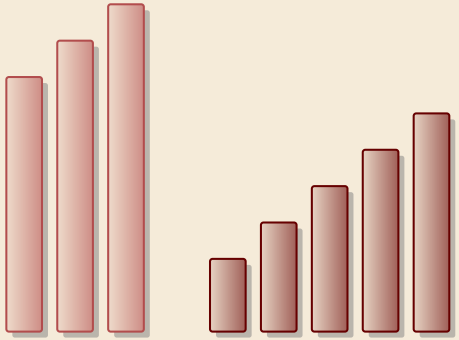
Merging sorted lists





Merging sorted lists

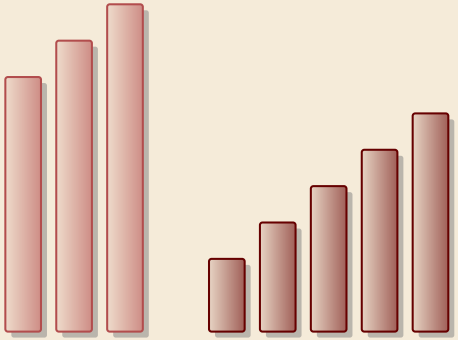




run1 run2 result

Merging sorted lists

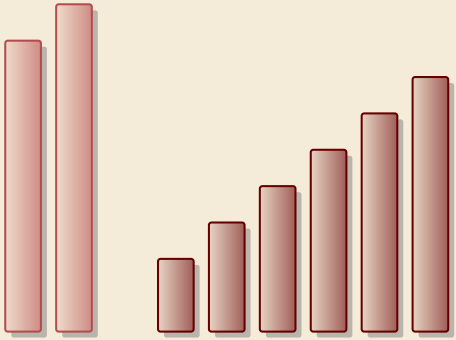




run1 run2 result

Merging sorted lists

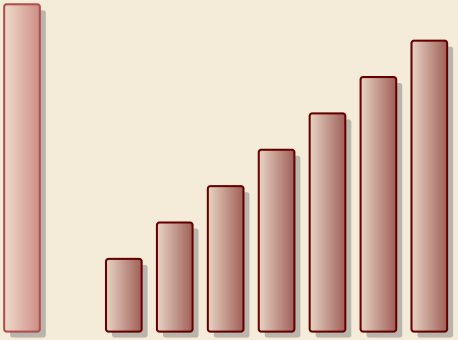




run1 run2 result

Merging sorted lists

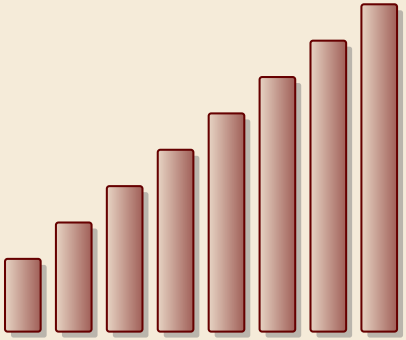




run1 run2 result

Merging sorted lists





run1 run2 result

Clicker Question

What is the worst-case running time of mergesort?



- **A** $\Theta(1)$
- **B** $\Theta(\log n)$
- **C** $\Theta(\log \log n)$
- **D** $\Theta(\sqrt{n})$
- **E** $\Theta(n)$
- **F** $\Theta(n \log \log n)$
- **G** $\Theta(n \log n)$
- **H** $\Theta(n \log^2 n)$
- **I** $\Theta(n^{1+\epsilon})$
- **J** $\Theta(n^2)$
- **K** $\Theta(n^3)$
- **L** $\Theta(2^n)$



→ *sli.do/effalg*

Clicker Question

What is the worst-case running time of mergesort?



A	~~$\Theta(1)$~~	G	$\Theta(n \log n)$ ✓
B	~~$\Theta(\log n)$~~	H	~~$\Theta(n \log^2 n)$~~
C	~~$\Theta(\log \log n)$~~	I	~~$\Theta(n^{1+\epsilon})$~~
D	~~$\Theta(\sqrt{n})$~~	J	~~$\Theta(n^2)$~~
E	~~$\Theta(n)$~~	K	~~$\Theta(n^3)$~~
F	~~$\Theta(n \log \log n)$~~	L	~~$\Theta(2^n)$~~



→ *sli.do/effalg*

Mergesort

mergesort(A[0..n))

```
1 procedure mergesort(A[l..r)):
2     n := r − l
3     if n ≤ 1 return
4     m := l + ⌊n/2⌋
5     mergesort(A[l..m))
6     mergesort(A[m..r))
7     merge(A[l..m), A[m..r), buf)
8     copy buf to A[l..r)
```

- recursive procedure
- merging needs
  - temporary storage *buf* for result (of same size as merged runs)
  - to read and write each element twice (once for merging, once for copying back)

Mergesort

```
1 procedure mergesort(A[l..r]):
2     n := r − l
3     if n ≤ 1 return
4     m := l + ⌊n/2⌋
5     mergesort(A[l..m])
6     mergesort(A[m..r])
7     merge(A[l..m], A[m..r], buf)   } Θ(n)
8     copy buf to A[l..r]            } Θ(n)
```

- recursive procedure
- merging needs
  - temporary storage *buf* for result (of same size as merged runs)
  - to read and write each element twice (once for merging, once for copying back)

Analysis: count *"element visits"* (read and/or write)

$$C(n) = \begin{cases} 0 & n \le 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 2n & n \ge 2 \end{cases}$$

same for best and worst case!

Simplification $n = 2^k$

$$C(2^k) = \begin{cases} 0 & k \le 0 \\ 2 \cdot C(2^{k-1}) + 2 \cdot 2^k & k \ge 1 \end{cases} = 2 \cdot 2^k + 2^2 \cdot 2^{k-1} + 2^3 \cdot 2^{k-2} + \cdots + 2^k \cdot 2^1 = 2k \cdot 2^k$$

2^{k+1} — 1 — k copies

$$C(n) = 2n \lg(n) = \Theta(n \log n)$$

(arbitrary n: $C(n) \le C(\text{next larger power of 2}) \le 4n \lg(n) + 4n = \Theta(n \log n)$)

Mergesort

```
1 procedure mergesort(A[l..r)):
2     n := r − l
3     if n ≤ 1 return
4     m := l + ⌊n/2⌋
5     mergesort(A[l..m))
6     mergesort(A[m..r))
7     merge(A[l..m), A[m..r), buf)
8     copy buf to A[l..r)
```

- recursive procedure
- merging needs
  - temporary storage *buf* for result (of same size as merged runs)
  - to read and write each element twice (once for merging, once for copying back)

Analysis: count *"element visits"* (read and/or write)

$$C(n) = \begin{cases} 0 & n \le 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 2n & n \ge 2 \end{cases}$$

same for best and worst case!

precisely(!) solvable *without* assumption $n = 2^k$:
$C(n) = 2n \lg(n) + \left(2 - \{\lg(n)\} - 2^{1-\{\lg(n)\}}\right)2n$
with $\{x\} := x - \lfloor x \rfloor$

Simplification $n = 2^k$

$$C(2^k) = \begin{cases} 0 & k \le 0 \\ 2 \cdot C(2^{k-1}) + 2 \cdot 2^k & k \ge 1 \end{cases} = 2 \cdot 2^k + 2^2 \cdot 2^{k-1} + 2^3 \cdot 2^{k-2} + \cdots + 2^k \cdot 2^1 = 2k \cdot 2^k$$

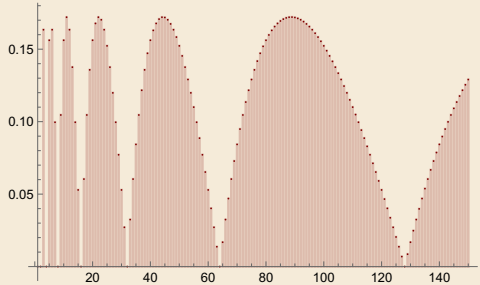
$C(n) = 2n \lg(n) = \Theta(n \log n)$ (arbitrary n: $C(n) \le C(\text{next larger power of 2}) \le 4n \lg(n) + 4n = \Theta(n \log n)$)

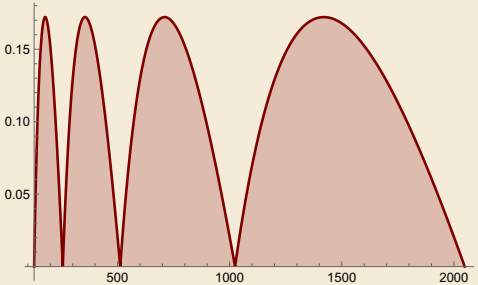
Linear Term of $C(n)$

Recall:

$$C(n) \;=\; 2n\lg(n) + \big(2 - \{\lg(n)\} - 2^{1-\{\lg(n)\}}\big)2n \qquad \text{with } \{x\} \coloneqq x - \lfloor x \rfloor$$

Plot of $2\big(2 - \{\lg(n)\} - 2^{1-\{\lg(n)\}}\big)$





$\rightsquigarrow$ Can prove: $C(n) \;\le\; 2n\lg n + 0.172n$

Mergesort – Discussion



optimal time complexity of $\Theta(n \log n)$ in the worst case



stable sorting method i. e., retains relative order of equal-key items



memory access is sequential (scans over arrays)



requires $\Theta(n)$ extra space

there are in-place merging methods,
but they are substantially more complicated
and not (widely) used

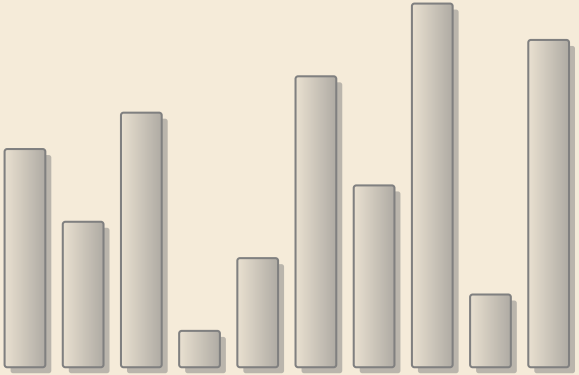
4.2 Quicksort

Partitioning around a pivot



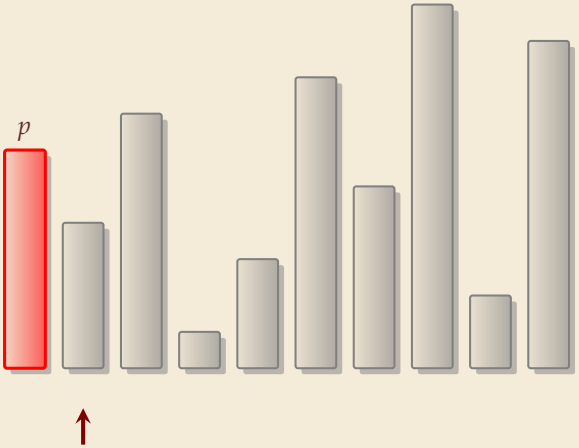
Partitioning around a pivot





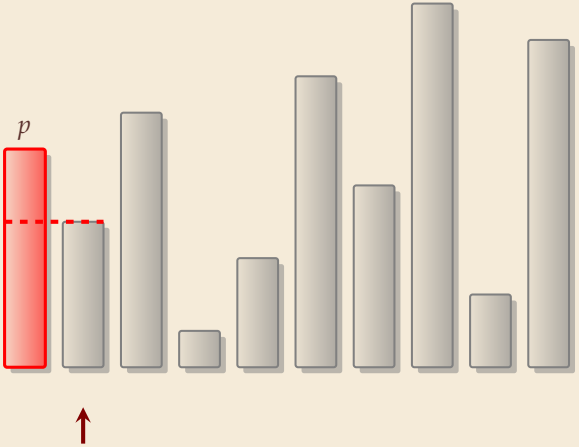
Partitioning around a pivot





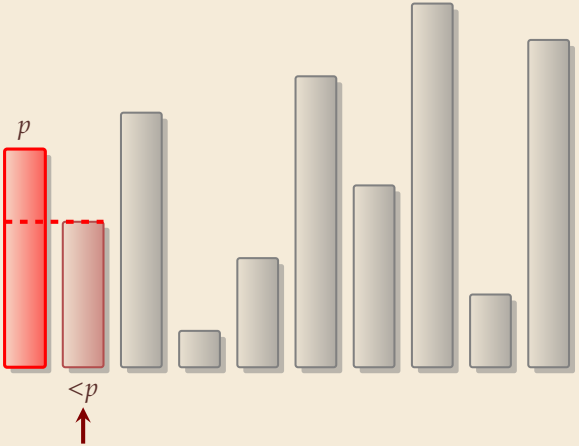
Partitioning around a pivot





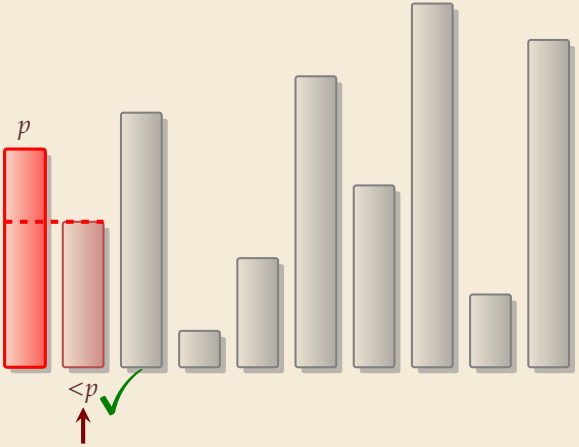
Partitioning around a pivot





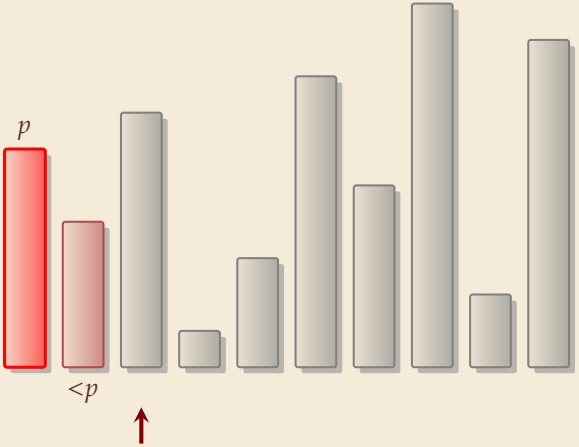
Partitioning around a pivot





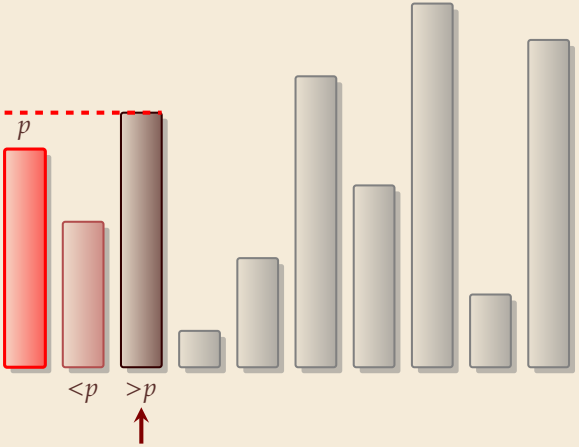
Partitioning around a pivot





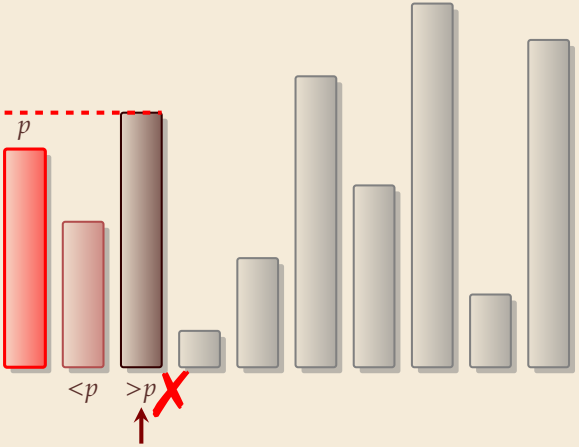
Partitioning around a pivot





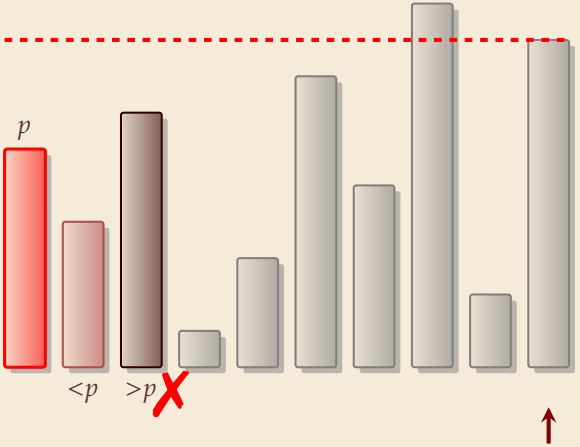
Partitioning around a pivot





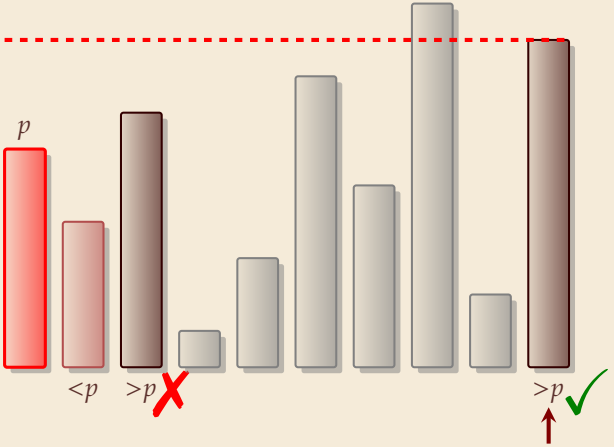
Partitioning around a pivot





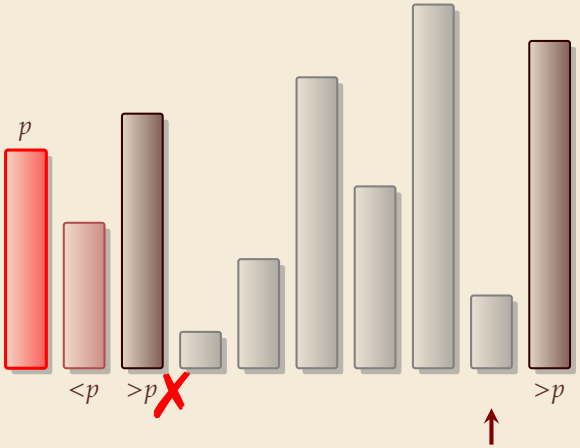
Partitioning around a pivot





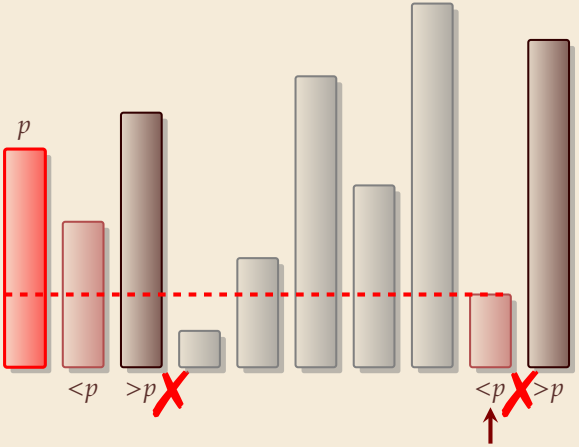
Partitioning around a pivot





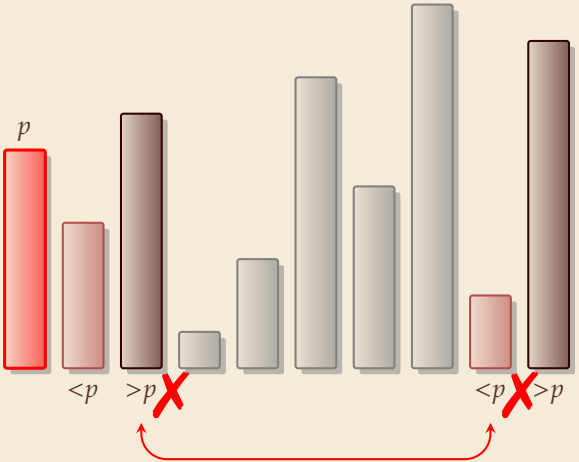
Partitioning around a pivot





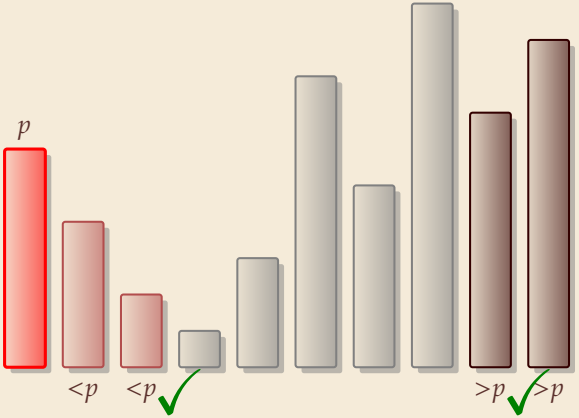
Partitioning around a pivot





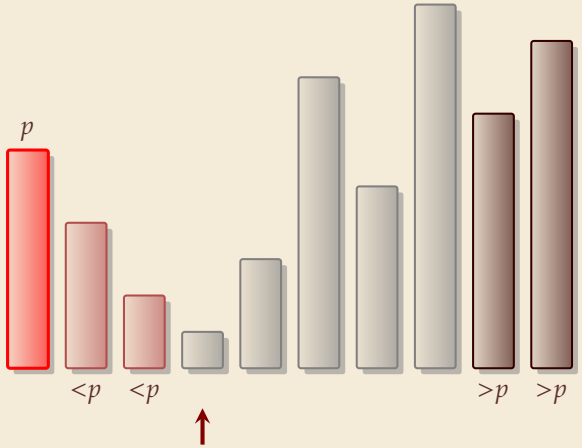
Partitioning around a pivot





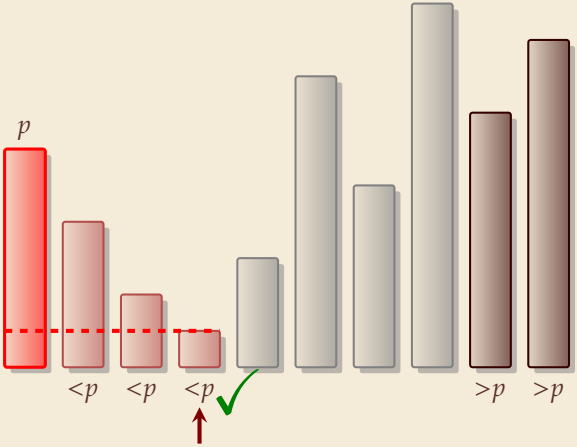
Partitioning around a pivot





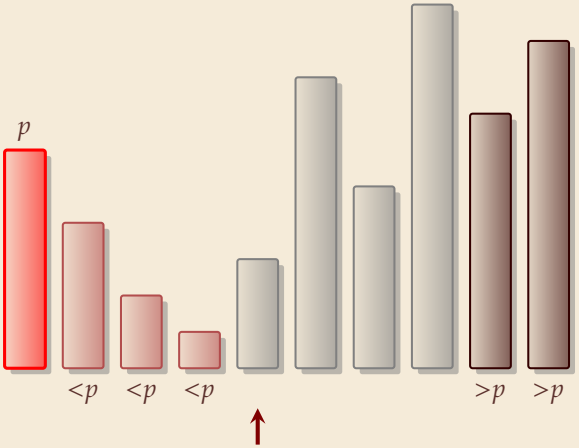
Partitioning around a pivot





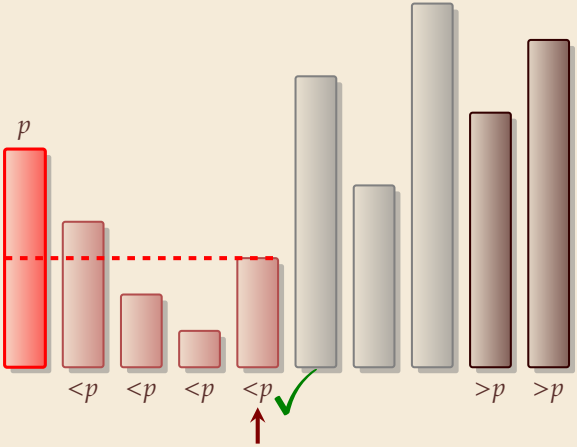
Partitioning around a pivot





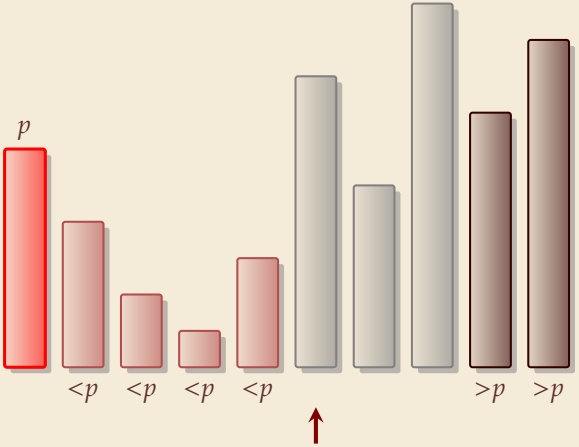
Partitioning around a pivot





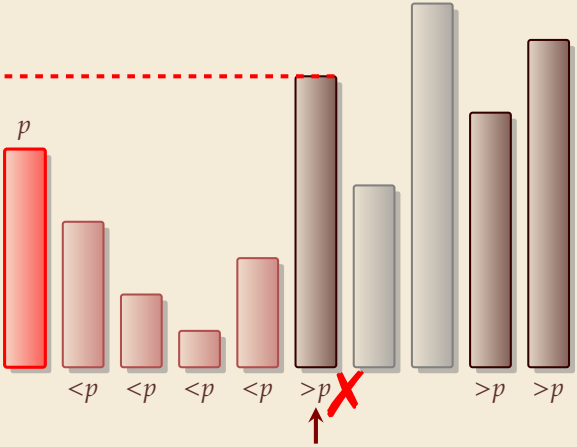
Partitioning around a pivot





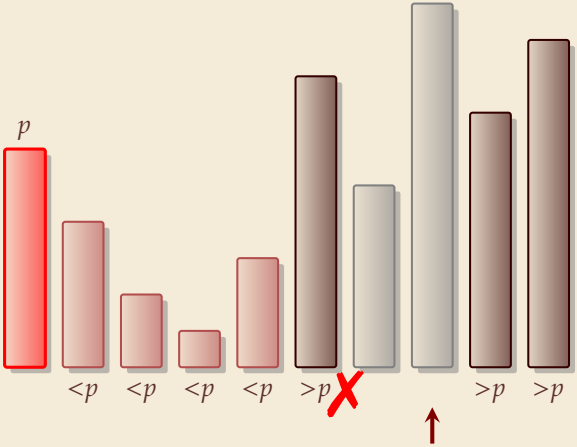
Partitioning around a pivot





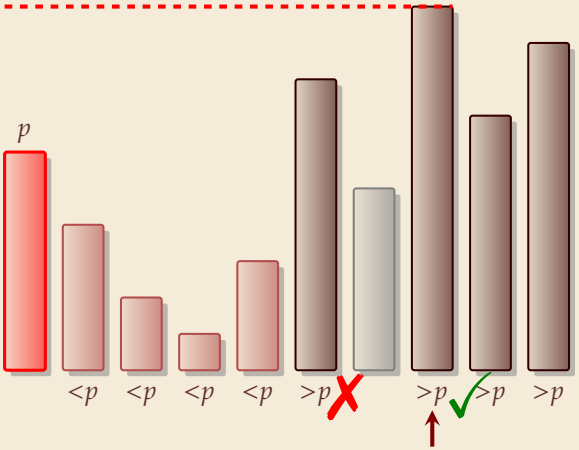
Partitioning around a pivot





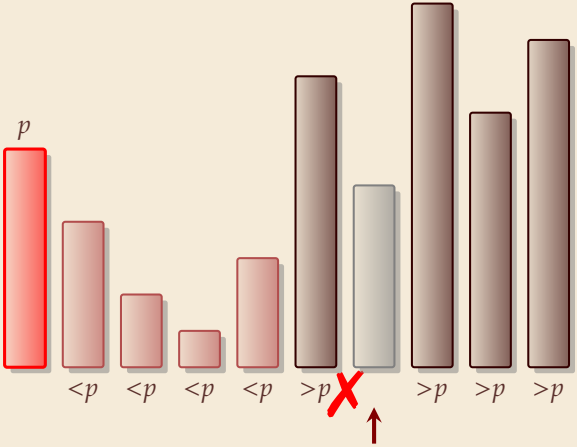
Partitioning around a pivot





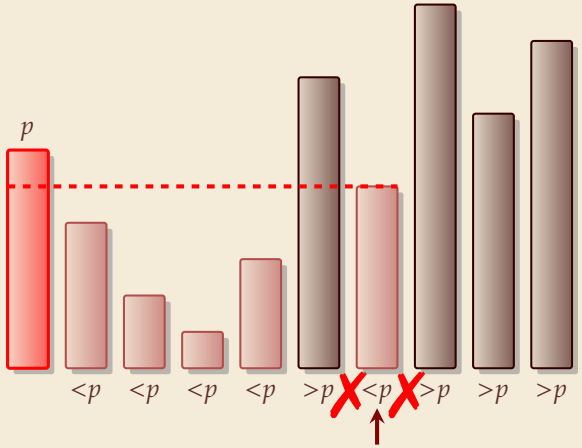
Partitioning around a pivot





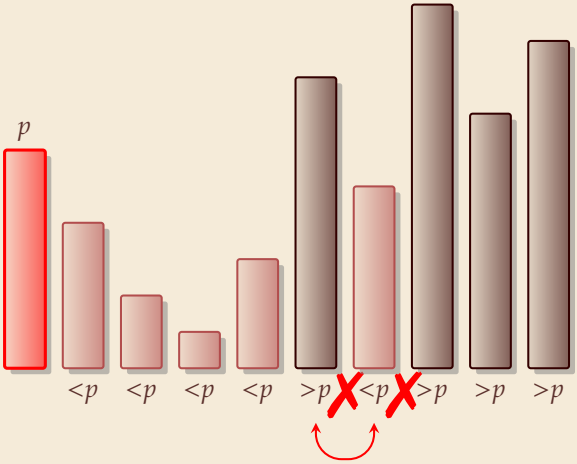
Partitioning around a pivot





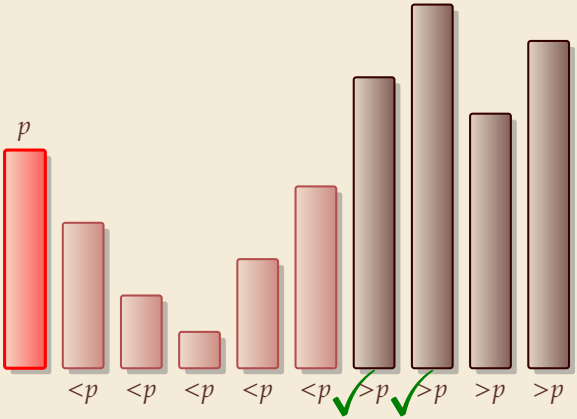
Partitioning around a pivot





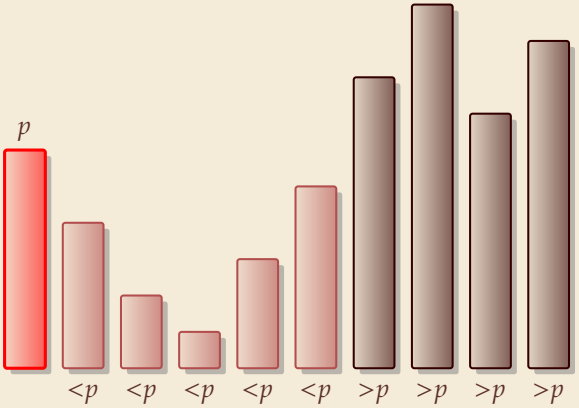
Partitioning around a pivot





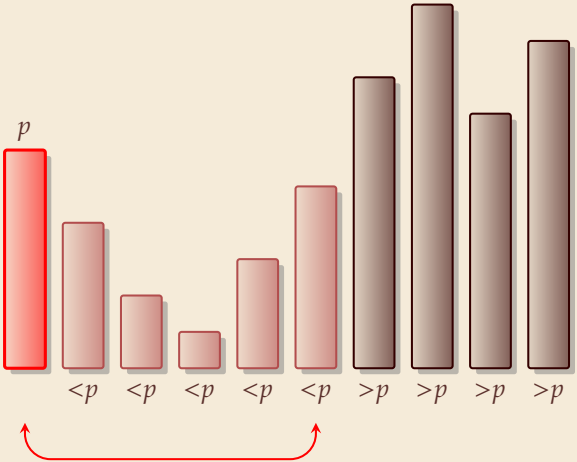
Partitioning around a pivot





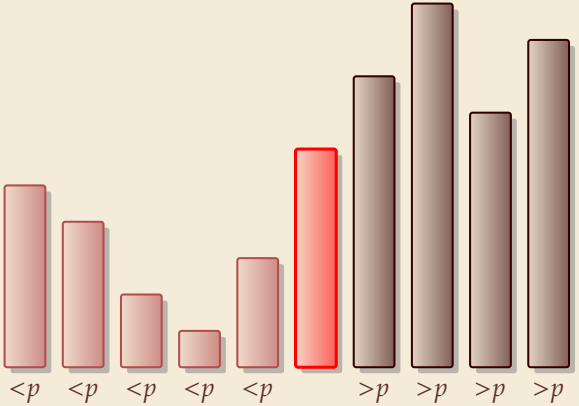
Partitioning around a pivot





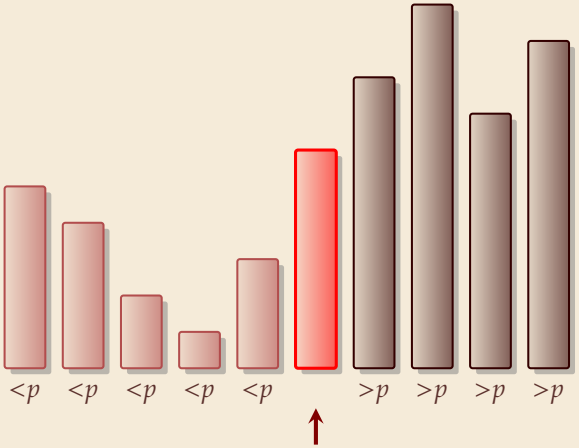
Partitioning around a pivot





Partitioning around a pivot





- no extra space needed
- visits each element once
- returns rank/position of pivot

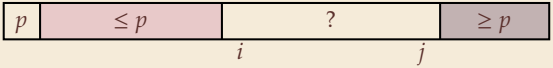
Partitioning – Detailed code

Beware: details easy to get wrong; use this code!

(if you ever have to)

```
1  procedure partition(A, b):
2      // input: array A[0..n), position of pivot b ∈ [0..n)
3      swap(A[0], A[b])
4      i := 0,    j := n
5      while true do
6          do i := i + 1 while i < n and A[i] < A[0]
7          do j := j − 1 while j ≥ 1 and A[j] > A[0]
8          if i ≥ j then break     (goto 11)
9          else swap(A[i], A[j])
10     end while
11     swap(A[0], A[j])
12     return j
```

Loop invariant (5–10): A



Quicksort

```
1 procedure quicksort(A[l..r)):
2     if r − l ≤ 1 then return
3     b := choosePivot(A[l..r))
4     j := l + partition(A[l..r), b)
5     quicksort(A[l..j))
6     quicksort(A[j + 1..r))
```

- recursive procedure
- choice of pivot can be
  - fixed position ⇝ dangerous!
  - random
  - more sophisticated, e. g., median of 3

Clicker Question

What is the worst-case running time of quicksort?



- **A** $\Theta(1)$
- **B** $\Theta(\log n)$
- **C** $\Theta(\log \log n)$
- **D** $\Theta(\sqrt{n})$
- **E** $\Theta(n)$
- **F** $\Theta(n \log \log n)$
- **G** $\Theta(n \log n)$
- **H** $\Theta(n \log^2 n)$
- **I** $\Theta(n^{1+\epsilon})$
- **J** $\Theta(n^2)$
- **K** $\Theta(n^3)$
- **L** $\Theta(2^n)$



→ *sli.do/effalg*

Clicker Question

What is the worst-case running time of quicksort?



A	~~$\Theta(1)$~~	G	~~$\Theta(n \log n)$~~
B	~~$\Theta(\log n)$~~	H	~~$\Theta(n \log^2 n)$~~
C	~~$\Theta(\log \log n)$~~	I	~~$\Theta(n^{1+\epsilon})$~~
D	~~$\Theta(\sqrt{n})$~~	J	$\Theta(n^2)$ ✓
E	~~$\Theta(n)$~~	K	~~$\Theta(n^3)$~~
F	~~$\Theta(n \log \log n)$~~	L	~~$\Theta(2^n)$~~



→ *sli.do/effalg*

Quicksort & Binary Search Trees

Quicksort



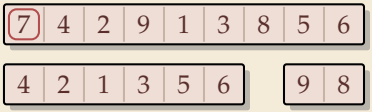
Quicksort & Binary Search Trees

Quicksort



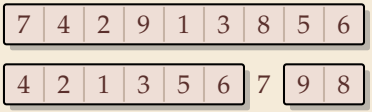
Quicksort & Binary Search Trees

Quicksort



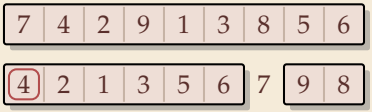
Quicksort & Binary Search Trees

Quicksort



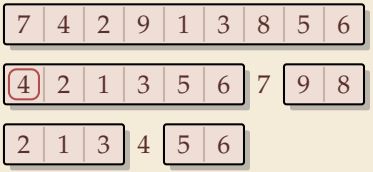
Quicksort & Binary Search Trees

Quicksort



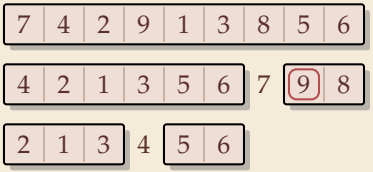
Quicksort & Binary Search Trees

Quicksort



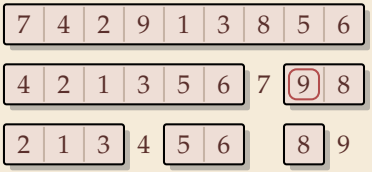
Quicksort & Binary Search Trees

Quicksort



Quicksort & Binary Search Trees

Quicksort

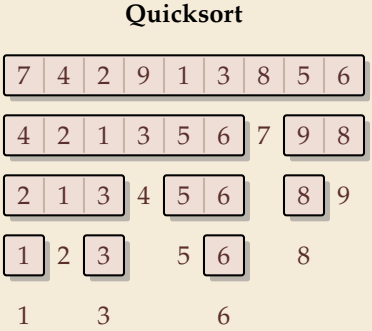


Quicksort & Binary Search Trees

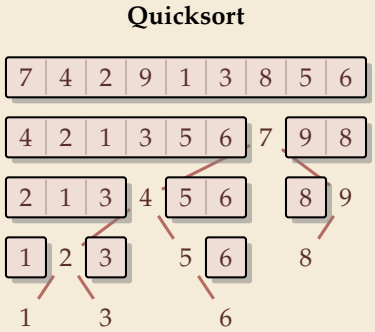
Quicksort



Quicksort & Binary Search Trees

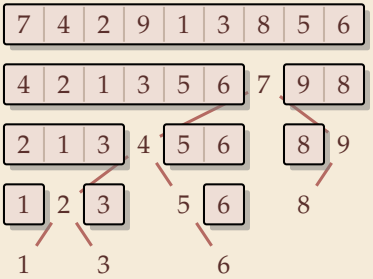


Quicksort & Binary Search Trees



Quicksort & Binary Search Trees

Quicksort

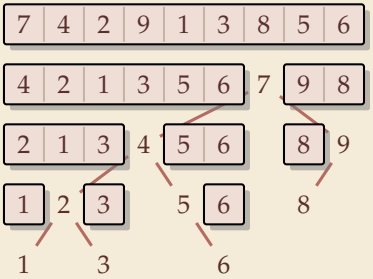


Binary Search Tree (BST)

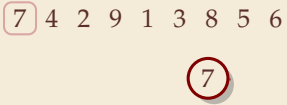
7 4 2 9 1 3 8 5 6

Quicksort & Binary Search Trees

Quicksort

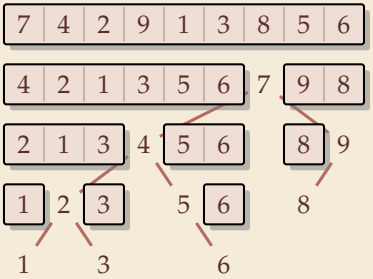


Binary Search Tree (BST)

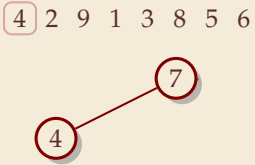


Quicksort & Binary Search Trees

Quicksort

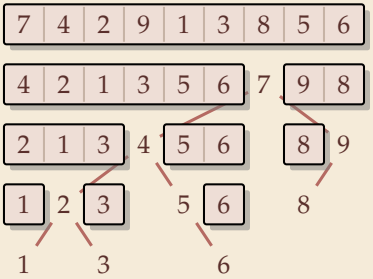


Binary Search Tree (BST)

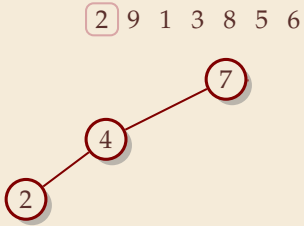


Quicksort & Binary Search Trees

Quicksort

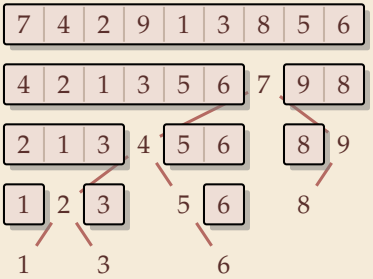


Binary Search Tree (BST)

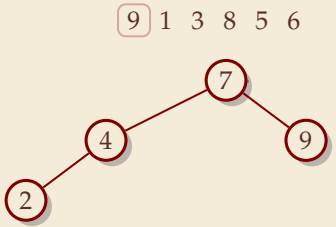


Quicksort & Binary Search Trees

Quicksort

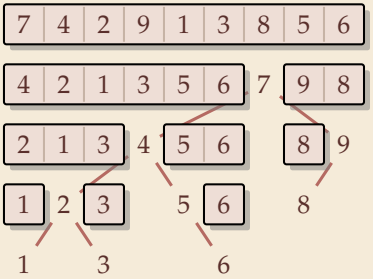


Binary Search Tree (BST)

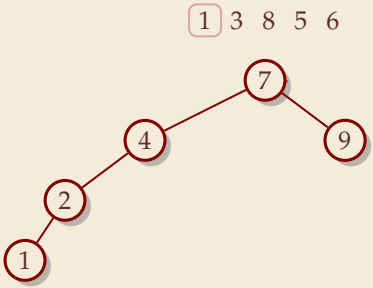


Quicksort & Binary Search Trees

Quicksort

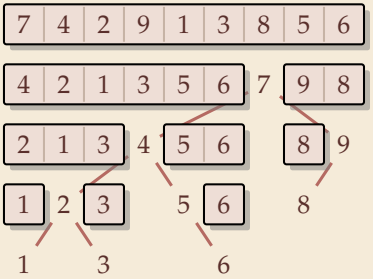


Binary Search Tree (BST)

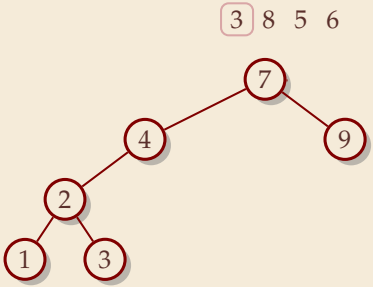


Quicksort & Binary Search Trees

Quicksort

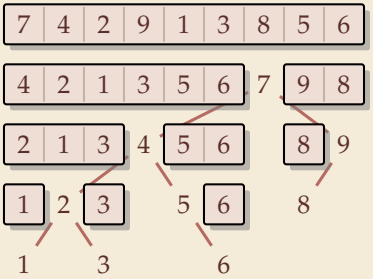


Binary Search Tree (BST)

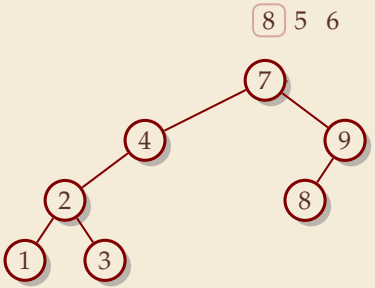


Quicksort & Binary Search Trees

Quicksort

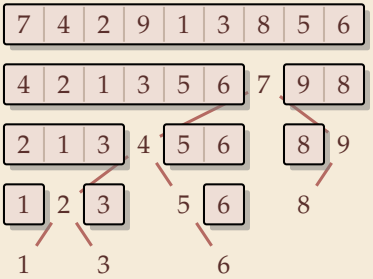


Binary Search Tree (BST)

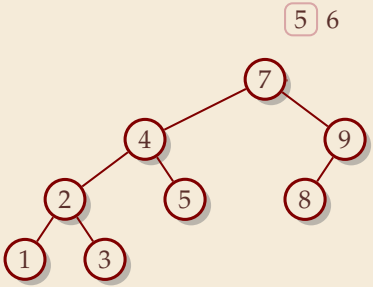


Quicksort & Binary Search Trees

Quicksort

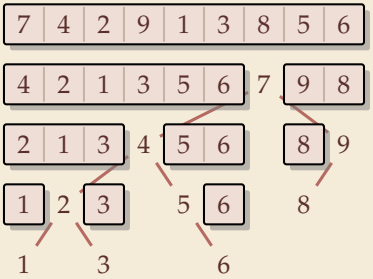


Binary Search Tree (BST)

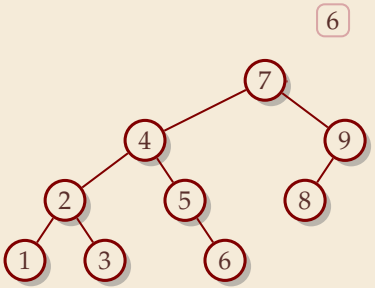


Quicksort & Binary Search Trees

Quicksort



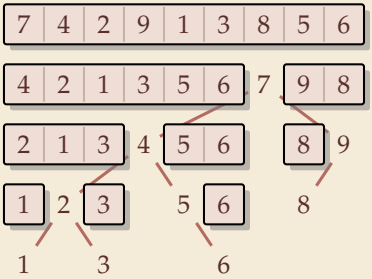
Binary Search Tree (BST)



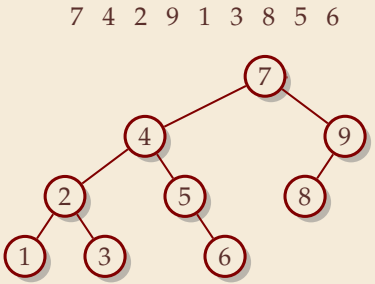
Quicksort & Binary Search Trees

(stable partitioning method)

Quicksort



Binary Search Tree (BST)

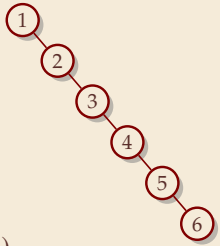


- recursion tree of quicksort = binary search tree from successive insertion
- comparisons in quicksort = comparisons to built BST
- comparisons in quicksort $\approx$ comparisons to search each element in BST

Quicksort – Worst Case

- Problem: BSTs can degenerate
- Cost to search for k is k

⇝ Total cost $\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \sim \frac{1}{2}n^2$



⇝ quicksort worst-case running time is in $\Theta(n^2)$

terribly slow!

But, we can fix this:

Randomized quicksort:

- choose a *random pivot* in each step

⇝ same as randomly *shuffling* input before sorting

Randomized Quicksort – Analysis

- cost measure: element visits (as for mergesort)
- $C(n)$ = #element visits when sorting n randomly permuted elements
  = cost of searching every element in BST built from input

Randomized Quicksort – Analysis

- cost measure: element visits (as for mergesort)
- $C(n)$ = #element visits when sorting n randomly permuted elements
  = cost of searching every element in BST built from input

$\rightsquigarrow$ quicksort needs $\sim 2\ln(2) \cdot n \lg n \approx 1.39 n \lg n$ *in expectation*
(see analysis of C_n in Unit 3!)

- also: very unlikely to be much worse:
  e. g., one can prove: $\Pr[\text{cost} > 10n \lg n] = O(n^{-2.5})$
  distribution of costs is "concentrated around mean"
  - even stronger bounds known  **McDiarmid, Hayward**: *Large Deviations for Quicksort*, J. Algorithms 21(3) **1996**
- intuition: have to be *constantly* unlucky with pivot choice

Quicksort – Discussion

- fastest general-purpose method
- $\Theta(n \log n)$ average case
- works *in-place* (no extra space required)
- memory access is sequential (scans over arrays)
- $\Theta(n^2)$ worst case (although extremely unlikely)
- not a *stable* sorting method

stable = elements equal with sorting order keep same relative ordering as in input

Open problem: Simple algorithm that is fast, stable and in-place.

4.3 Comparison-Based Lower Bound

Lower Bounds

- **Lower bound:** mathematical proof that *no algorithm* can do better.
  - very powerful concept: bulletproof *impossibility* result
    ≈ *conservation of energy* in physics
  - **(unique?) feature of computer science:**
    for many problems, solutions are known that (asymptotically) **achieve the lower bound**

  ⇝ can speak of "*optimal* algorithms"

Lower Bounds

- **Lower bound:** mathematical proof that *no algorithm* can do better.
  - very powerful concept: bulletproof *impossibility* result
    ≈ *conservation of energy* in physics
  - **(unique?) feature of computer science:**
    for many problems, solutions are known that (asymptotically) **achieve the lower bound**

  ⇝ can speak of "*optimal* algorithms"

- To prove a statement about *all algorithms*, we must precisely define what that is!
- already know one option: the word-RAM model
- Here: use a simpler, more restricted model.

The Comparison Model

- In the *comparison model* data can only be accessed in two ways:
  - comparing two elements
  - moving elements around (e. g. copying, swapping)
  - Cost: number of comparisons.

The Comparison Model

- In the *comparison model* data can only be accessed in two ways:
  - comparing two elements
  - moving elements around (e. g. copying, swapping)
  - Cost: number of comparisons.

That's good!
Keeps algorithms general!

- This makes very few assumptions on the kind of objects we are sorting.
- Mergesort and Quicksort work in the comparison model.

The Comparison Model

- In the *comparison model* data can only be accessed in two ways:
  - comparing two elements
  - moving elements around (e. g. copying, swapping)
  - Cost: number of comparisons.

That's good!
Keeps algorithms general!

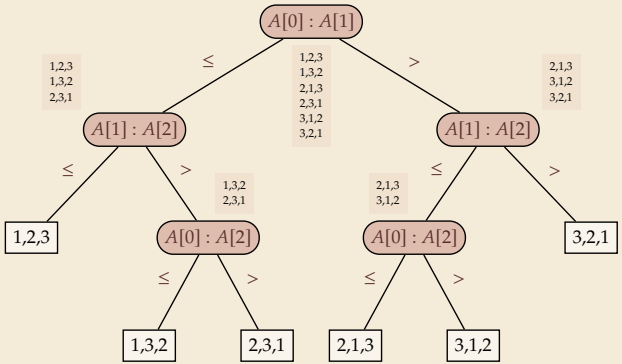
- This makes very few assumptions on the kind of objects we are sorting.
- Mergesort and Quicksort work in the comparison model.

⇝ Every comparison-based sorting algorithm corresponds to a ***decision tree***.

- only model comparisons ⇝ ignore data movement
- nodes = comparisons the algorithm does
- child links = outcomes of comparison
- leaf = unique initial input permutation compatible with comparison outcomes
- next comparisons can depend on outcomes ⇝ child subtrees can look different

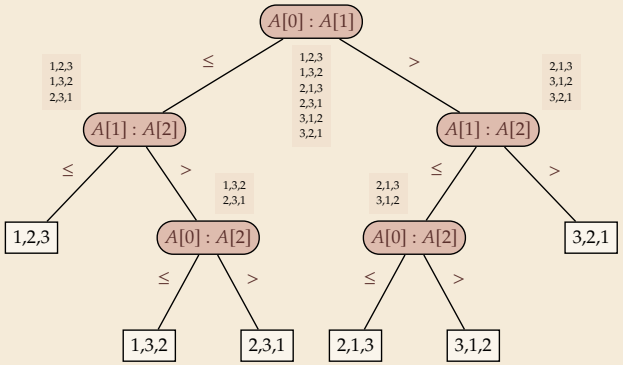
Comparison Lower Bound

Example: Comparison tree for a sorting method for $A[0..2]$:



Comparison Lower Bound

Example: Comparison tree for a sorting method for $A[0..2]$:



- Execution = follow a path in comparison tree.

⇝ height of comparison tree = worst-case # comparisons

- comparison trees are *binary* trees

⇝ ℓ leaves ⇝ height $\geq \lceil \lg(\ell) \rceil$

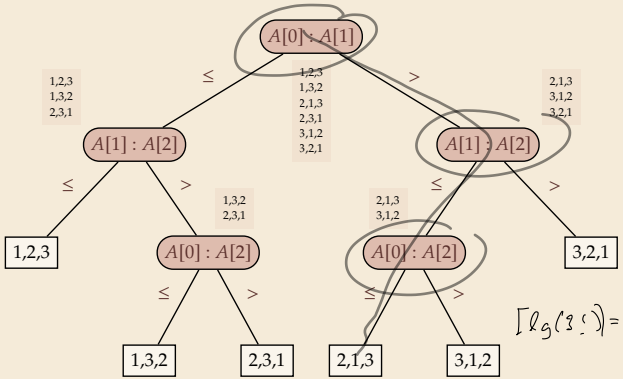
- comparison trees for sorting method must have $\geq n!$ leaves

⇝ height $\geq \lg(n!) \sim n \lg n$

more precisely: $\lg(n!) = n \lg n - \lg(e)n + O(\log n)$

Comparison Lower Bound

Example: Comparison tree for a sorting method for $A[0..2]$:



$\lceil \lg(3!) \rceil = \lceil \lg(6) \rceil = 3$

- Execution = follow a path in comparison tree.
  - ⇝ height of comparison tree = worst-case # comparisons
- comparison trees are *binary* trees
  - ⇝ ℓ leaves ⇝ height $\geq \lceil \lg(\ell) \rceil$
- comparison trees for sorting method must have $\geq n!$ leaves
  - ⇝ height $\geq \lg(n!) \sim n \lg n$

more precisely: $\lg(n!) = n \lg n - \lg(e)n + O(\log n)$

- Mergesort achieves $\sim n \lg n$ comparisons ⇝ asymptotically comparison-optimal!

Clicker Question



Does the comparison-tree from the previous slide correspond to a worst-case optimal sorting method?

A Yes

B No



→ *sli.do/effalg*

Clicker Question



Does the comparison-tree from the previous slide correspond to a worst-case optimal sorting method?

A Yes ✓

B ~~No~~



→ *sli.do/effalg*

How tight is the lower bound?

∉ exam

Worst-case #comparisons to sort n elements:

	comparisons	$\approx n \lg n - c \cdot n$
Lower bound $\lceil \lg n! \rceil$	$n \lg n - n \lg e + O(\log n)$	$c = \lg e \approx 1.443$
Mergesort (top-down)	$n\lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$	$c \in [0.91, 1]$
Binary insertion (#cmps only!)	$\sum_{k=1}^{n} \lceil \lg k \rceil = n\lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$	the same!
Merge-insertion (Ford–Johnson)	$n \lg n - (3 - \lg 3)\, n + O(\log n)$ (for best n)	$c \in [1.329, 1.415]$
Group-insertion (Sergeev 2020/21)	$\lg n! + o(n)$	$c \to \lg e$

How tight is the lower bound?

Worst-case #comparisons to sort n elements:

	comparisons	$\approx n \lg n - c \cdot n$
Lower bound $\lceil \lg n! \rceil$	$n \lg n - n \lg e + O(\log n)$	$c = \lg e \approx 1.443$
Mergesort (top-down)	$n\lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$	$c \in [0.91, 1]$
Binary insertion (#cmps only!)	$\sum_{k=1}^{n} \lceil \lg k \rceil = n\lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$	the same!
Merge-insertion (Ford–Johnson)	$n \lg n - (3 - \lg 3)\, n + O(\log n)$ (for best n)	$c \in [1.329, 1.415]$
Group-insertion (Sergeev 2020/21)	$\lg n! + o(n)$	$c \longrightarrow \lg e$

- Merge-insertion is *exactly* optimal for $n \leq 15$ and $n = 20, 21$, but *not* for $n = 47$ (Manacher 1979).
- In 2020, Sergeev announced an algorithm with $\lg n! + o(n)$ comparisons
  long and intricate proof, only asymptotically better
- the $O(n)$ gap is irrelevant for running time – but it is the oldest question about sorting.

Ford, Johnson: *A Tournament Problem*, Amer. Math. Monthly 66(5) **1959**

Manacher: *The Ford–Johnson Sorting Algorithm Is Not Optimal*, J. ACM 26(3) **1979**

Sergeev: *On the Upper Bound of the Complexity of Sorting*, Comput. Math. Math. Phys. 61(2) / ECCC TR20-096 **2021**

4.4 Integer Sorting

Clicker Question

Select all **correct formulations** of our **lower bound** from §4.3.



A Any sorting algorithm requires $O(n \log n)$ running time in the worst case.

B Every comparison-based sorting algorithm requires $\Omega(n \log n)$ running time in the worst case for sorting n elements.

C Every comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case for sorting n elements.

D Every sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case for sorting n elements.

E The complexity of sorting n elements in the comparison-model is $\Theta(n \log n)$.

F The complexity of sorting n elements in the comparison-model is $\Omega(n \log n)$.



→ *sli.do/effalg*

Clicker Question

Select all **correct formulations** of our **lower bound** from §4.3.



A ~~Any sorting algorithm requires $O(n \log n)$ running time in the worst case.~~

B Every comparison-based sorting algorithm requires $\Omega(n \log n)$ running time in the worst case for sorting n elements. ✓

C Every comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case for sorting n elements. ✓

D ~~Every sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case for sorting n elements.~~

E ~~The complexity of sorting n elements in the comparison-model is $\Theta(n \log n)$.~~

F The complexity of sorting n elements in the comparison-model is $\Omega(n \log n)$. ✓



→ *sli.do/effalg*

How to beat a lower bound

- Does the above lower bound mean, sorting always takes time $\Omega(n \log n)$?

How to beat a lower bound

- Does the above lower bound mean, sorting always takes time $\Omega(n \log n)$?

- **Not necessarily;** only in the *comparison model!*
  - $\rightsquigarrow$ Lower bounds show where to *change* the model!

How to beat a lower bound

- Does the above lower bound mean, sorting always takes time $\Omega(n \log n)$?

- **Not necessarily;** only in the *comparison model!*
  - ⇝ Lower bounds show where to *change* the model!

- Here: sort n **integers**
  - can do *a lot* with integers: add them up, compute averages, ... (full power of word-RAM)
  - ⇝ we are **not** working in the comparison model
  - ⇝ *above lower bound does not apply!*

How to beat a lower bound

- Does the above lower bound mean, sorting always takes time $\Omega(n \log n)$?

- **Not necessarily;** only in the *comparison model!*
  - ⇝ Lower bounds show where to *change* the model!

- Here: sort n **integers**
  - can do *a lot* with integers: add them up, compute averages, . . . (full power of word-RAM)
  - ⇝ we are **not** working in the comparison model
  - ⇝ *above lower bound does not apply!*

  - but: a priori unclear how much arithmetic helps for sorting . . .

Counting sort

- Important parameter: size/range of numbers
  - numbers in range $[0..U) = \{0, \ldots, U-1\}$ typically $U = 2^b$ $\rightsquigarrow$ b-bit binary numbers

Counting sort

- Important parameter: size/range of numbers
  - numbers in range $[0..U) = \{0, \ldots, U-1\}$ typically $U = 2^b$ $\rightsquigarrow$ b-bit binary numbers
- We can sort n integers in $\Theta(n+U)$ time and $\Theta(U)$ space when $b \leq w$:

word size

Counting sort

```
1  procedure countingSort(A[0..n)):
2      // A contains integers in range [0..U).
3      C[0..U) := new integer array, initialized to 0
4      // Count occurrences
5      for i := 0,...,n-1
6          C[A[i]] := C[A[i]] + 1
7      i := 0 // Produce sorted list
8      for k := 0,...U-1
9          for j := 1,...C[k]
10             A[i] := k; i := i + 1
```

Θ(n)

all inner loops combined Θ(n)

- *count* how often each *possible* value occurs
- produce sorted result directly from counts
- circumvents lower bound by using integers as array index / pointer offset

$\rightsquigarrow$ Can sort n integers in range $[0..U)$ with $U = O(n)$ in time and space $\Theta(n)$.

Larger Universes: Radix Sort

- ***MSD Radix Sort:***
  - split numbers into base-R "digits"
  - Use counting sort on most significant digit
    (with variant of counting sort that moves full number)
  - $\rightsquigarrow$ integers sorted with respect to first digit
  - recurse on sublist for each digit value, using next digit for counting sort

$\rightsquigarrow$ After $\lfloor \log_R(U) \rfloor + 1$ levels of counting sort, fully sorted!

- For $R = O(n)$ (hence $R \leq 2^w$), all counting sort calls on the same level cost total of $O(n)$ time
  (requires care: reuse C across the small subproblems instead of re-initializing it)

$\rightsquigarrow$ total time $O(n \log_R(U)) = O\left(n \frac{\log(U)}{\log(R)}\right)$

$\rightsquigarrow$ $O(n)$ time sorting possible for numbers in range $U = O(n^c)$ for constant c.

Integer Sorting – State of the art

∉ exam

Algorithm theory

- integer sorting on the w-bit word-RAM
- suppose $U = 2^w$, but w can be an arbitrary function of n
- how fast can we sort n such w-bit integers on a w-bit word-RAM?
  - for $w = O(\log n)$: linear time (*radix/counting sort*)
  - for $w = \Omega(\log^{2+\varepsilon} n)$: linear time (*signature sort*)
  - for w in between: can do $O(n\sqrt{\lg \lg n})$ (randomized, very complicated algorithm)
    don't know if that is best possible!

Andersson, Hagerup, Nilsson, Raman: *Sorting in Linear Time?*, J. Comput. Syst. Sci. 57(1) **1998**

Han, Thorup: *Integer Sorting in $O(n\sqrt{\log \log n})$ Expected Time and Linear Space*, FOCS **2002**

Han: *Deterministic Sorting in $O(n \log \log n)$ Time and Linear Space*, J. Algorithms 50(1) **2004**

Integer Sorting – State of the art

Algorithm theory

- integer sorting on the w-bit word-RAM
- suppose $U = 2^w$, but w can be an arbitrary function of n
- how fast can we sort n such w-bit integers on a w-bit word-RAM?
  - for $w = O(\log n)$: linear time (*radix/counting sort*)
  - for $w = \Omega(\log^{2+\varepsilon} n)$: linear time (*signature sort*)
  - for w in between: can do $O(n\sqrt{\lg \lg n})$ (randomized, very complicated algorithm)
    don't know if that is best possible!

Andersson, Hagerup, Nilsson, Raman: *Sorting in Linear Time?*, J. Comput. Syst. Sci. 57(1) **1998**

Han, Thorup: *Integer Sorting in $O(n\sqrt{\log \log n})$ Expected Time and Linear Space*, FOCS **2002**

Han: *Deterministic Sorting in $O(n \log \log n)$ Time and Linear Space*, J. Algorithms 50(1) **2004**

* * *

. . . for the rest of this unit: back to the comparisons model!

Clicker Question



Which statements are correct? Select all that apply.

My computer has 64-bit words, so an `int` has 64 bits. Hence I can sort any `int[]` of length n ...

- **A** in constant time.
- **B** in $O(\log n)$ time.
- **C** in $O(n)$ time.
- **D** in $O(n \log n)$ time.
- **E** some time, but not possible to say from given information.



→ *sli.do/effalg*

Clicker Question



Which statements are correct? Select all that apply.

My computer has 64-bit words, so an `int` has 64 bits. Hence I can sort any `int[]` of length n ...

- **A** ~~in constant time.~~
- **B** ~~in $O(\log n)$ time.~~
- **C** in $O(n)$ time. ✓
- **D** in $O(n \log n)$ time. ✓
- **E** some time, but not possible to say from given information. ✓



→ *sli.do/effalg*

Part II

Exploiting presortedness

4.5 Adaptive Sorting

Adaptive sorting

- Comparison lower bound also holds for the *average case* $\rightsquigarrow$ $\lg(n!)$ cmps necessary
- Mergesort and Quicksort from above use $\Theta(n \lg n)$ cmps even in best case

Adaptive sorting

- Comparison lower bound also holds for the *average case* $\rightsquigarrow$ $\lg(n!)$ cmps necessary
- Mergesort and Quicksort from above use $\Theta(n \lg n)$ cmps even in best case



Can we do better if the input is already "almost sorted"?

Adaptive sorting

- Comparison lower bound also holds for the *average case* $\rightsquigarrow$ $\lg(n!)$ cmps necessary
- Mergesort and Quicksort from above use $\Theta(n \lg n)$ cmps even in best case



Can we do better if the input is already "almost sorted"?

Scenarios where this may arise naturally:

- Append new data as it arrives, regularly sort entire list (e. g., log files, database tables)
- Compute summary statistics of time series of measurements that change slowly over time (e. g., weather data)
- Merging locally sorted data from different servers (e. g., map-reduce frameworks)

$\rightsquigarrow$ Ideally, algorithms should *adapt* to input: *the more sorted the input, the faster the algorithm*

. . . but how to do that!?

Warmup: check for sorted inputs

- Any method could first check if input already completely in order!
  - 👍 Best case becomes $\Theta(n)$ with $n - 1$ comparisons!
  - 👎 Usually $n - 1$ extra comparisons and pass over data "wasted"
  - 👎 Only catches a single, extremely special case ...

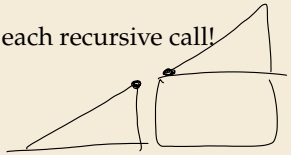
Warmup: check for sorted inputs

- Any method could first check if input already completely in order!
  - 👍 Best case becomes $\Theta(n)$ with $n - 1$ comparisons!
  - 👎 Usually $n - 1$ extra comparisons and pass over data "wasted"
  - 👎 Only catches a single, extremely special case ...
- For divide & conquer algorithms, could check in each recursive call!
  - 👍 Potentially exploits partial sortedness!
  - 👎 usually adds $\Omega(n \log n)$ extra comparisons

Warmup: check for sorted inputs

- Any method could first check if input already completely in order!
  - Best case becomes $\Theta(n)$ with $n - 1$ comparisons!
  - Usually $n - 1$ extra comparisons and pass over data "wasted"
  - Only catches a single, extremely special case ...

- For divide & conquer algorithms, could check in each recursive call!
  - Potentially exploits partial sortedness!
  - usually adds $\Omega(n \log n)$ extra comparisons



For Mergesort, can instead check before merge with a **single** comparison

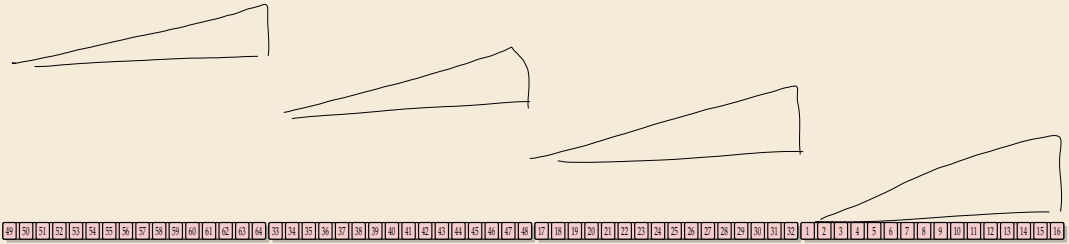
- If last element of first run $\leq$ first element of second run, skip merge

How effective is this idea?

```
1  procedure mergesortCheck(A[l..r)):
2      n := r − l
3      if n ≤ 1 return
4      m := l + ⌊n/2⌋
5      mergesortCheck(A[l..m))
6      mergesortCheck(A[m..r))
7      if A[m − 1] > A[m]
8          merge(A[l..m), A[m..r), buf)
9          copy buf to A[l..r)
```

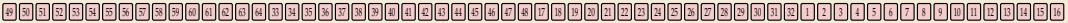
Mergesort with sorted check – Analysis

- Simplified cost measure: *merge cost* = size of output of merges
  ≈ number of comparisons
  ≈ number of memory transfers / cache misses
- Example input: $n = 64$ numbers in sorted *runs* of 16 numbers each:



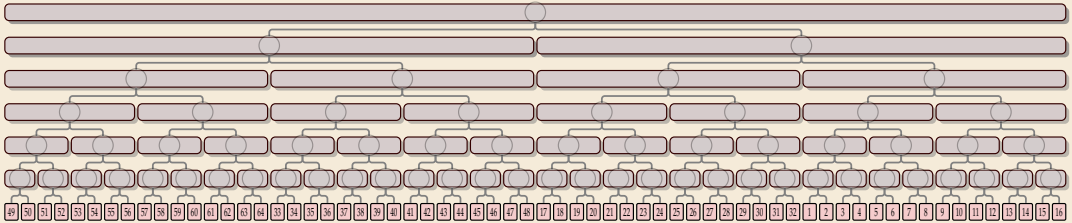
Mergesort with sorted check – Analysis

- Simplified cost measure: *merge cost* = size of output of merges
  ≈ number of comparisons
  ≈ number of memory transfers / cache misses
- Example input: $n = 64$ numbers in sorted *runs* of 16 numbers each:



Mergesort with sorted check – Analysis

- Simplified cost measure: *merge cost* = size of output of merges
  ≈ number of comparisons
  ≈ number of memory transfers / cache misses
- Example input: $n = 64$ numbers in sorted *runs* of 16 numbers each:



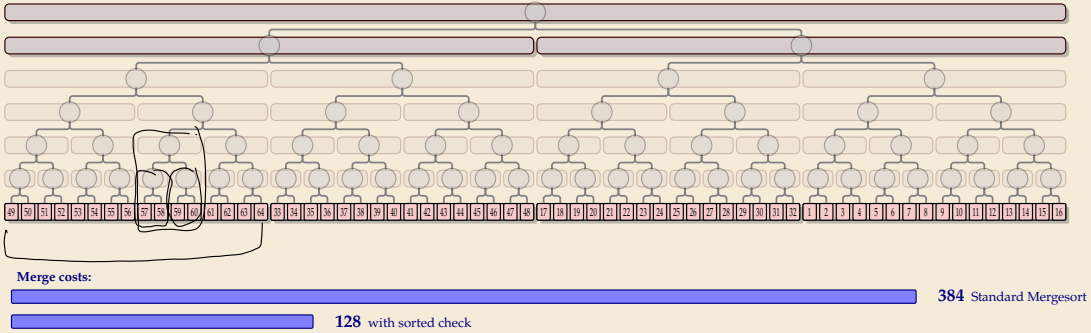
Merge costs:



384 Standard Mergesort

Mergesort with sorted check – Analysis

- Simplified cost measure: *merge cost* = size of output of merges
  ≈ number of comparisons
  ≈ number of memory transfers / cache misses
- Example input: $n = 64$ numbers in sorted *runs* of 16 numbers each:



Sorted check can help a lot!

Alignment issues

- In previous example, each run of length ℓ saved us $\ell \lg(\ell)$ in merge cost.
  - = exactly the cost of *creating* this run in mergesort had it not already existed
  - ⇝ best savings we can hope for!

⇝ Are overall merge costs $\mathcal{H}(\ell_1, \ldots, \ell_r) := \underbrace{n \lg(n)}_{\text{mergesort}} - \underbrace{\sum_{i=1}^{r} \ell_i \lg(\ell_i)}_{\text{savings from runs}}$?

(ℓ_i = length of ith run; $n = \sum \ell_i$)

$$= -\sum_{i=1}^{r} \left[\ell_i \lg(\ell_i) - \ell_i \lg n \right]$$

$$= \sum_{i=1}^{r} \ell_i \lg\left(\frac{n}{\ell_i}\right)$$

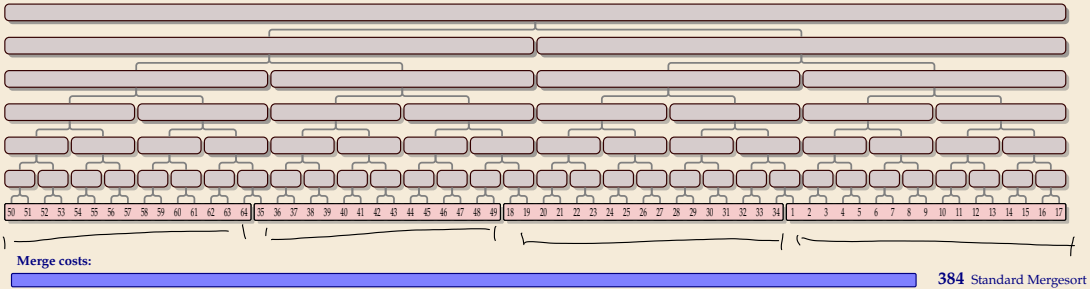
Alignment issues

- In previous example, each run of length ℓ saved us $\ell \lg(\ell)$ in merge cost.
  - = exactly the cost of *creating* this run in mergesort had it not already existed
  - $\rightsquigarrow$ best savings we can hope for!

$\rightsquigarrow$ Are overall merge costs $\mathcal{H}(\ell_1, \ldots, \ell_r) := \underbrace{n \lg(n)}_{\text{mergesort}} - \underbrace{\sum_{i=1}^{r} \ell_i \lg(\ell_i)}_{\text{savings from runs}}$?

ℓ_i = length of ith run

Unfortunately, not quite:



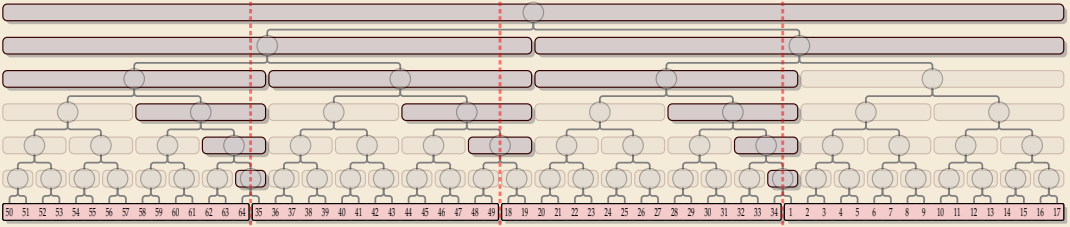
Alignment issues

- In previous example, each run of length ℓ saved us $\ell \lg(\ell)$ in merge cost.
  - = exactly the cost of *creating* this run in mergesort had it not already existed
  - ⇝ best savings we can hope for!

⇝ Are overall merge costs $\mathcal{H}(\ell_1, \ldots, \ell_r) := \underbrace{n \lg(n)}_{\text{mergesort}} - \underbrace{\sum_{i=1}^{r} \ell_i \lg(\ell_i)}_{\text{savings from runs}}$?

(ℓ_i = length of ith run)

Unfortunately, not quite:



Merge costs:

384 Standard Mergesort

216 with sorted check

127.8 $\mathcal{H}(15, 15, 17, 17)$